STRATA

Browse in context.

DOC 03 · TECHNICAL REQUIREMENTS DOCUMENT

Technical Requirements Document

Architecture, stack, native modules, the Critical IPC security rule, Continuum internals, and performance targets.

Version: 1.0

Date: September 2026

Status: Draft for team review

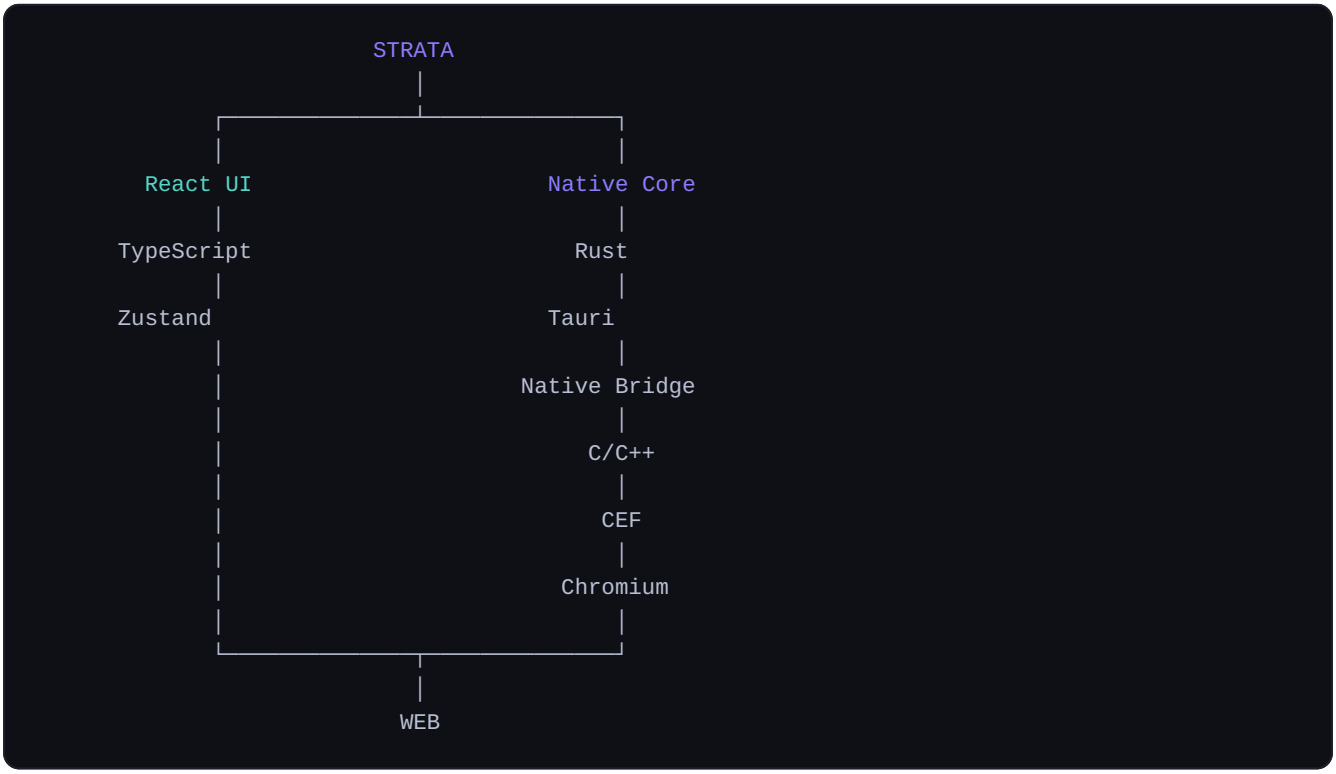
Owner: Engineering

CONTENTS

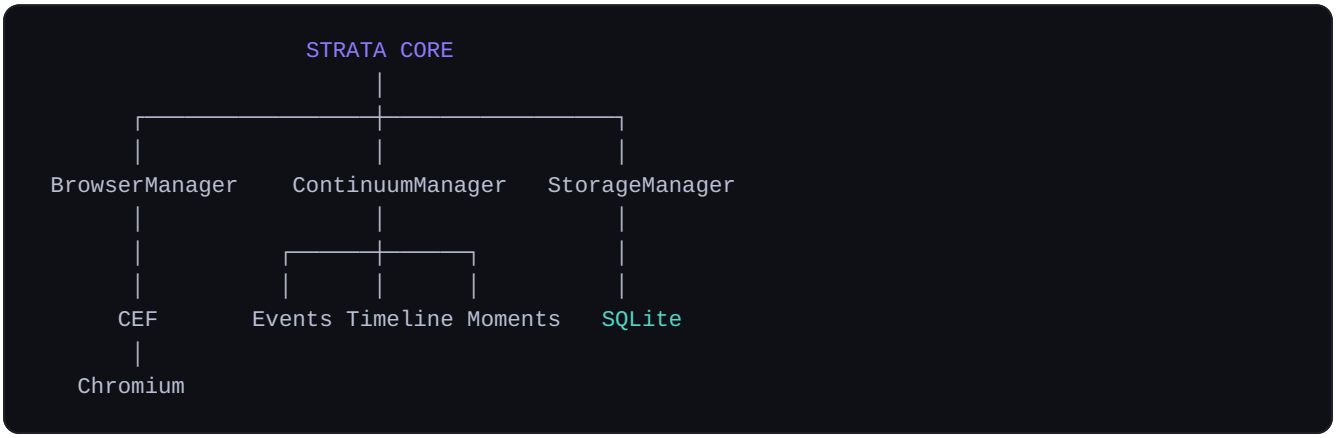
01 · ARCHITECTURE

System Architecture

Strata is a Chromium-based desktop browser built with a React/Tauri/Rust application layer and the Chromium Embedded Framework (CEF) as the embedded browser engine. The browser chrome is React; website content is rendered by Chromium via CEF — the Tauri WebView is never used to render arbitrary websites.



Internal core



BIGGEST TECHNICAL RISK

Not React, not Rust — it's **CEF + native window embedding + Rust/Tauri integration + Chromium lifecycle management**. CEF is a multi-process, callback-heavy C/C++ API with its own message-loop and browser/renderer process split. A tiny proof of concept (create browser → load URL → display in a native window → navigate → close) must work reliably before any React UI is built.

02 · STACK

Technology Stack

LAYER	TECHNOLOGY	RESPONSIBILITY
Browser chrome (frontend)	React, TypeScript, Vite, Tailwind CSS, Zustand, Framer Motion, Lucide	Tabs, address bar, menus, Continuum UI, Moments, settings, history, bookmarks, command palette
Desktop application layer	Rust + Tauri (TAO for windows, WRY for OS webviews)	Native application shell, window management, IPC between web frontend and Rust backend
Browser engine	Chromium Embedded Framework (CEF)	Embeds Chromium for actual page rendering, navigation, frames, JS integration, browser lifecycle
Native bridge	C/C++ (CEF is fundamentally a C/C++ API)	Connects Rust to CEF's browser-process and subprocess APIs (`CefBrowser` , `CefFrame` , `CefBrowserHost`)
Local data	SQLite	Fully local storage for Continuum events, Moments, bookmarks, downloads, settings

EXPLICIT CONSTRAINT

Do not attempt to force the entire browser engine into pure Rust. The native browser integration must use the appropriate CEF interfaces, in C/C++, bridged into Rust — not reimplemented.

Core Rust Modules

BrowserManager	SRC-TAURI/SRC/BROWSER/MANAGER.RS
`create_browser()`	Instantiate a CEF browser instance for a tab
`close_browser()`	Tear down a browser instance
`navigate() / reload() / stop()`	Navigation control
`go_back() / go_forward()`	History navigation

TabManager	SRC-TAURI/SRC/BROWSER/TAB.RS
`create_tab() / close_tab() / activate_tab()`	Tab lifecycle
`duplicate_tab() / move_tab()`	Tab manipulation
`pin_tab() / mute_tab()`	Tab state flags

WindowManager

`create_window()`, `close_window()`, `maximize()`, `minimize()`, `fullscreen()`, `resize()`

NavigationManager

Tracks URL, title, favicon, navigation start/completion, back/forward state, and redirects for every frame.

ContinuumManager

The most important Strata-specific component, composed of six sub-systems:

```
ContinuumManager
├── EventRecorder      records browser events
├── StateCollector     collects browser-owned page state
├── TimelineManager    powers the Rewind view
├── MomentManager      save / freeze / restore / delete / rename
├── SnapshotManager    periodic checkpoints
└── RestoreManager     reconstructs a workspace from events + checkpoints
```

EventRecorder

Records discrete browser events rather than continuous state:

```
{
  "type": "navigation",
  "tab_id": "tab_42",
  "url": "https://example.com/docs",
  "timestamp": 1788354920
}
```

StateCollector — PageState

```
PageState
├── url, title, favicon
├── scroll_x, scroll_y, zoom
├── navigation_index
├── tab_id, window_id
├── split_position
└── timestamp

Best-effort only:
├── selected_text
├── focused_element
├── page_state
└── serialized_form_state
```

Snapshot strategy

Continuum uses an event-oriented model with periodic checkpoints rather than saving every micro-interaction.

```
09:01  TAB_CREATED       Scroll event
09:02  NAVIGATION             ↓
09:03  SCROLL            Debounce 500–1000ms
09:05  NAVIGATION             ↓
09:08  SPLIT_CREATED     State changed?
09:12  ZOOM_CHANGED           ↓
09:15  MOMENT_SAVED      Record latest state
```

- Debounce scroll events; record meaningful state changes, not every pixel.
- Create periodic checkpoints so reconstruction never has to replay the entire event log.
- Explicit Moments are saved immediately, bypassing the debounce window entirely.

Security Architecture

Chromium is intentionally multi-process: web content is isolated into renderer processes while browser functionality lives in privileged browser-side processes, with sandboxing restricting a compromised renderer. Strata must not weaken this model for integration convenience. The developer preserves:

Process isolation

Renderer isolation, sandboxing, origin isolation

Access control

Permission boundaries, secure IPC

Data isolation

Profile isolation across Personal / Work / Development / Guest

Critical IPC rule

NEVER ALLOW

```
Website JavaScript
       ↓
   Tauri IPC
       ↓
Rust filesystem     × forbidden — direct path
```

Instead, every native operation is mediated by the CEF/Chromium sandbox and a controlled, explicitly authorized native API surface:

```
Website
   ↓
CEF / Chromium sandbox
   ↓
CEF browser process
   ↓
Controlled native API
   ↓
Rust
```

Only explicitly authorized operations cross the process boundary. The Rust/Tauri layer must never expose arbitrary native commands to webpages — every Tauri command handler is an allowlisted, purpose-built function, never a generic "run this" bridge.

Profiles & private mode

Profiles

Personal, Work, Development, Guest — each with separate cookies, history, local storage, bookmarks, downloads, Continuum data, and settings.

Private mode

No persistent history, Continuum, Moments, or cookies after the session ends. Some temporary in-memory state may exist while the window stays open.

Performance & Non-Functional Requirements

TARGET	GOAL
Startup	< 2 seconds on a reasonably modern machine, after optimization
UI frame rate	60 FPS+ for all browser chrome animations
Tab switching	Should feel immediate — no perceptible delay
Continuum recording	Negligible UI impact; all writes asynchronous and batched
Database	All Continuum writes async/batched; reads must not block the UI thread

Additional non-functional requirements: crash recovery (a crashed renderer process must not take down the whole application, per Chromium's process-isolation model), accessibility (full keyboard navigation, see UI/UX Brief), and storage growth management (event log periodically compacted into checkpoints, see Backend Schema §5).

Storage Philosophy

```
User's machine
      │
      ▼
 ~/.strata/
      │
      ├── strata.db
      ├── cache/
      ├── sessions/
      └── moments/
```

Everything is local-only by default: no account, no server, no cloud, no analytics requirement. Continuum data never leaves the machine unless the user explicitly enables a future synchronization feature. Full schema detail is in the Backend Schema document.

Project Structure & Platform Strategy

```
strata/
├── frontend/            React chrome (components, stores, hooks, lib, types)
├── src-tauri/            Rust core (browser, continuum, storage, downloads, permissions, comman
├── cef/                  CEF include/lib/resources/binaries
├── native/cef_bridge/    C/C++ browser/renderer/app process glue + CMakeLists.txt
├── scripts/
└── docs/
```

Recommended build order: Windows first. Get the browser working properly on one platform before attempting three — CEF's per-platform binary distribution and native windowing differences make simultaneous three-OS development a major source of schedule risk for a small team.